BIG DATA & CLOUD COMPUTING / PROJECT COMPANION

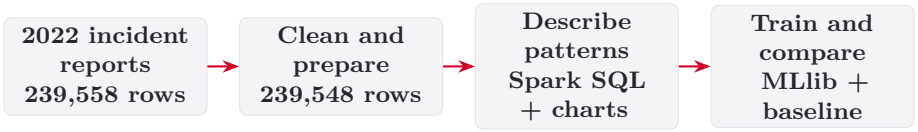
A Beginner's Guide to the Notebook

Predicting Recorded Arrest Outcomes in Chicago

Data analysis, Spark, charts, machine learning, and responsible interpretation

Start Here: What the Notebook Does

The notebook analyzes 2022 Chicago crime incident reports using Databricks and Apache Spark. It first describes how recorded arrest rates vary, then compares three Spark MLlib classifiers with a simple historical-rate lookup. The target is the dataset's **Arrest** field: whether an arrest was recorded for an incident.



11.7% overall recorded-arrest rate **0.844** GBT AUC-ROC **0.885** lookup AUC-ROC

The central distinction The notebook predicts a *recorded historical outcome*, not crime, guilt, solvability, future public safety, or whether an arrest should occur. Association is not causation.

A Quick Mental Model

A row is one reported incident. A column is one recorded attribute, such as crime type, location, district, or arrest status. The notebook's two questions are:

1. **Descriptive:** How do incident counts and recorded arrest rates differ across categories?
2. **Predictive:** Can available incident fields rank reports by recorded arrest outcome better than a simple rule?

The notebook is an analysis workflow, not a deployment system. It does not ingest live reports, recommend an action, or test whether a policing intervention improves safety.

Notebook Map and Technology

Cells	Stage	What happens
0–5	Setup and source	Imports libraries, creates a Spark session, and loads the Databricks table.
6–24	Data quality	Checks identifiers, parses timestamps, cleans location fields, counts missing data, and fills or drops selected rows.
25–64	Exploratory analysis	Counts categories; compares arrest rates by offense, time, district, location, and domestic context; runs Spark SQL.
65–74	Feature preparation	Builds date/time fields, screens associations, encodes categories, creates a feature vector, weights classes, and splits train/test data.
75–85	Models and comparisons	Fits three classifiers, calculates class metrics, builds a smoothed lookup baseline, plots cumulative gains, and checks subgroups.
86–89	Interpretation	Summarizes findings, recommendations, limitations, and responsible-use safeguards.

Libraries: who does what?

Library	Role in this notebook
PySpark SQL / DataFrames	Read and transform distributed tables; group, filter, aggregate, and query the incident records.
Spark MLlib	Index categorical fields, assemble features, train Decision Tree / Random Forest / GBT, and evaluate predictions.
Pandas	Hold small aggregated tables and format results after bringing them to the notebook driver with `toPandas()`.
NumPy	Array calculations for Cramer's V and cumulative-gain coordinates.
Matplotlib	Create axes, bars, lines, scatter plots, labels, and titles.
Seaborn	Draw readable statistical charts, including bar plots, heatmaps, and confusion matrices.
Python `math`	Calculate the domestic/non-domestic two-proportion z statistic.

Small code-reading note `OneHotEncoder`, `StringType`, and `os` are imported but not used in the final modeling path. The actual model uses `StringIndexer` plus `VectorAssembler`; it does not one-hot encode the features.

Spark DataFrame: a useful distinction

Many DataFrame operations are **transformations**: they describe a new table but are evaluated lazily. Examples are `withColumn`, `filter`, `drop`, and `groupBy`. Actions such as `count`, `first`, `collect`, and Databricks `display` trigger work and return results. `toPandas()` collects data onto the driver, so the notebook uses it mainly for compact summaries and plots, not to convert all 239,548 raw rows at once.

Data Source and Cleaning

Loading the table

`spark.table("crime_prediction_in_chicago")` reads an existing Spark table in the Databricks workspace. The notebook includes a commented CSV alternative, but the normal run assumes the table has already been uploaded. `SparkSession.builder...getOrCreate()` connects to or creates the Spark entry point used by Spark SQL and MLlib.

What was cleaned, and why?

Field / issue	Action	Reason and interpretation
Year	Drop	The supplied data cover 2022 only, so Year has no variation.
Updated On	Drop	Administrative update timestamp, not a planned predictor.
Case Number	Drop	239,511 distinct values for 239,558 rows; nearly an identifier and not a meaningful generalizable feature.
ID	Keep for trace, drop for modeling	Unique incident key; retained during cleaning but excluded from the feature set.
Date	Parse with `to_timestamp`	Converts text such as month/day/year and time into a timestamp Spark can query.
Block	Strip leading address number	`regexp_replace` keeps a generalized street name instead of a highly specific street number.
Coordinates / Location	Drop	Exact coordinate-related fields are missing in 4,667 rows (about 1.9%); they would require imputation or row loss. Other geography remains.
Ward	Drop 10 rows	Only 10 incidents lack a Ward, so these rows are removed.
Location Description	Fill 965 blanks	Replace missing entries with the mode, `STREET`, the most frequent value (about 0.4% of rows).

Functions behind the cleaning

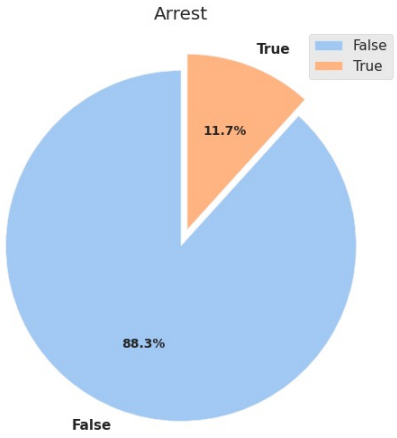
- `select`, `distinct`, `count`: choose a column, count unique values, and measure table size. This reveals that IDs are nearly unique.
- `printSchema` and `dtypes`: inspect column names and Spark data types before modeling.
- `withColumn` and `to_timestamp`: create or replace a column and parse its text values into timestamps.
- `regexp_replace`: apply a regular-expression substitution to standardize Block.
- `drop`, `dropna`, `na.fill`: remove columns, remove rows missing selected values, or fill nulls.
- `when`, `sum`, `alias`: count nulls by creating a 1/0 indicator, summing it, and naming the result.
- `groupBy`, `orderBy`, `head`: rank the most frequent location descriptions and find the fill value.

Data-quality caution The analysis keeps broader administrative geography such as District, Ward, Beat, and Block after removing exact coordinates. These fields may reflect historical reporting and enforcement patterns; cleaning them does not remove that interpretive risk.

Exploratory Analysis I: What Is in the Dataset?

Frequency plots

Cells 28–35 repeat one pattern: `groupBy(category).count()`, sort descending, keep the top ten, convert the small result to Pandas, then call Seaborn `barplot`. They show counts, not arrest probabilities. The plots cover IUCR codes, full crime descriptions, Primary Type, FBI code, and Location Description.



The target distribution: 88.3% False and 11.7% True.

Useful observations

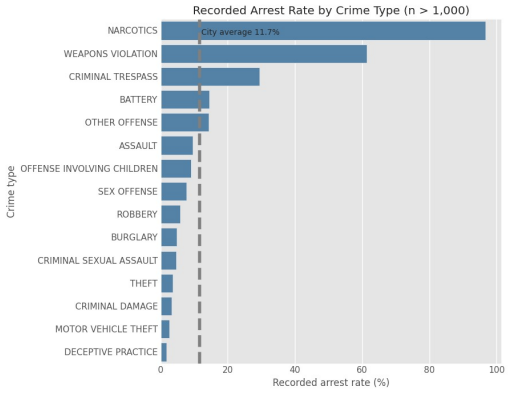
- There are 31 Primary Type categories. Theft (54,884), Battery (40,945), and Criminal Damage (27,248) are the largest; together they account for 51.4% of reports.
- The top three location labels are Street (28.6%), Apartment (19.1%), and Residence (12.8%).
- Full Description has about 285 distinct values; IUCR and FBI Code largely encode related crime classifications.
- A top-ten chart is a readability choice: categories outside the top ten still exist in the data.

The target and a descriptive field

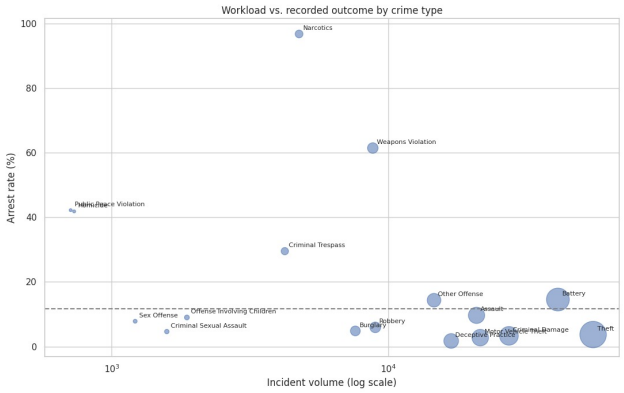
The pie chart groups rows by `Arrest`: 28,050 of 239,548 incidents have a recorded arrest (11.7%); 211,498 do not (88.3%). `Domestic` is another binary field, not the target: 44,916 reports (18.8%) are flagged domestic. A pie chart communicates a share, but exact counts and rates are better for detailed comparisons.

Why imbalance matters A classifier that always predicts "no arrest" is correct about 88% of the time while finding zero arrests. Later pages therefore distinguish majority-class accuracy from arrest-class precision and recall.

Exploratory Analysis II: Offense Type and Workload



Recorded arrest rate by Primary Type; dashed line is the city average.



Workload versus outcome; bubble size is volume and x-axis is logarithmic.

How to read these charts

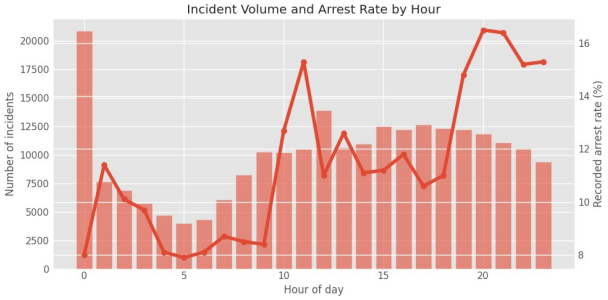
The rate chart calculates 100 × arrests/incidents for each crime type, excludes categories with fewer than 1,000 incidents, and sorts rates from high to low. The dashed vertical line marks the citywide 11.7% average. Filtering small groups reduces unstable rates caused by tiny denominators.

The bubble chart uses one point per crime type: x-position is incident count on a **log scale**, y-position is recorded arrest rate, and bubble size also grows with count. The log scale lets both common and rare categories fit on the same view. A large point far right means high workload; a high point means a high recorded-arrest rate. Neither axis explains why.

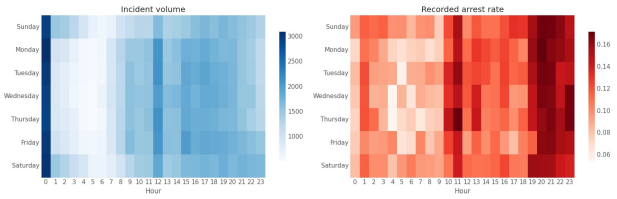
Incident group	Reports	Rate	Reading
Narcotics	4,755	96.7%	Very high rate, relatively small volume.
Weapons violations	8,779	61.4%	High rate, moderate volume.
Criminal trespass	4,223	29.5%	Above the city average.
Four property categories	120,447	3.2%	About half of all reports, with a low recorded-arrest rate.

Interpretation guardrail The 3.2% property-crime rate identifies a large workload/outcome gap. The notebook cannot say that the gap is caused by police performance, case quality, victim cooperation, or any single factor.

Exploratory Analysis III: Time and Calendar



Hour: bars show incident count; line shows arrest rate.



Day-by-hour heatmaps: count on left, rate on right.

Spark date functions

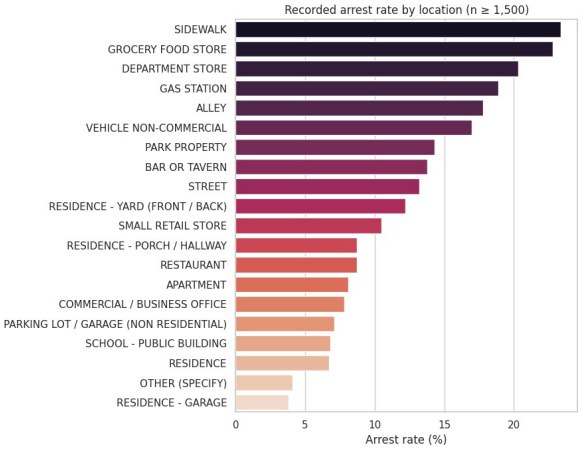
The notebook derives `hour`, `day_of_week`, and `month` with `F.hour`, `F.dayofweek`, and `F.month`. It groups by each field, counts incidents, and averages the 0/1 Arrest value. For a binary indicator, the average is the proportion of rows with value 1; multiplying by 100 gives a percentage.

- **Hour:** 04:00–06:00 is about 7.9–8.1%; 19:00–23:00 is about 14.8–16.5%. The volume bar at midnight is suspicious: 20,874 rows at hour 0 versus 7,630 at hour 1. Many unknown times may have been stored as 00:00.
- **Weekday:** arrest rates are nearly flat, from 11.0% Friday to 12.3% Sunday. Day of week is a weak descriptive signal.
- **Month:** May is highest at 14.3%; October–November are lowest at about 9.7–9.8%.

Chart grammar. In dual-axis charts, bars use the left axis for volume and the line uses the right axis for arrest rate. The two scales are different, so compare trends within each series, not the height of a bar directly to the line. The heatmap uses color intensity to show a value in each weekday/hour cell; the left panel is volume and the right panel is rate.

Caution The 00:00 spike is a data-quality warning, not evidence of a true midnight crime peak. Calendar patterns are descriptive and may be distorted by missing or default timestamps.

Exploratory Analysis IV: Location and Domestic Context



Location arrest rates; chart includes locations with at least 1,500 reports.

Main location results

- Sidewalk: 23.4% (12,059 reports)
- Grocery store: 22.8%
- Gas station: 18.9%
- Residence: 6.7%; apartment: 8.1%

The `n >= 1500` filter avoids highlighting rates based on very small groups. These are recorded associations. One possible explanation is that a suspect may be present at some public/commercial incidents, but the data does not test that mechanism.

Domestic comparison and the z-test

The notebook aggregates reports and arrests separately for Domestic=True and False, then calculates each group's rate. Domestic incidents have a 12.6% rate (44,916 reports), versus 11.5% for other incidents (194,632 reports). A two-proportion z-test gives $z \approx 6.4$, so the difference is statistically significant in this large sample. Its size is only 1.1 percentage points, however; statistical significance does not equal practical importance or causation.

For two proportions, the notebook pools the observed rate p, estimates the standard error as

$$\sqrt{p(1-p)\left(\frac{1}{n_1}+\frac{1}{n_2}\right)},$$

then divides the observed rate difference by that error. `math.erfc` converts the z statistic to a two-sided normal-tail p-value. This simple test assumes independent observations; repeated reports from the same case may weaken that assumption.

Districts, Spark SQL, and Proactive Offenses

Why raw district rates mislead

The raw district rate is the number of recorded arrests divided by reports in that district. It ranges from 7.6% (District 19) to 23.2% (District 11), roughly a threefold gap. Districts do not receive the same mix of crime types.

The notebook estimates a citywide arrest rate for each Primary Type, joins those rates back to incidents, sums the expected rate across each district's mix, and computes:

$$\text{observed/expected ratio} = \frac{\text{actual arrests in district}}{\text{expected arrests from its offense mix}}.$$

The ratio ranges from 0.80 (District 3) to 1.40 (District 1). A value of 1.00 means observed and expected counts match under this simple crime-mix calculation. It is an adjustment, not a causal or complete fairness analysis and not a police-performance ranking.

Spark SQL repeats a DataFrame analysis

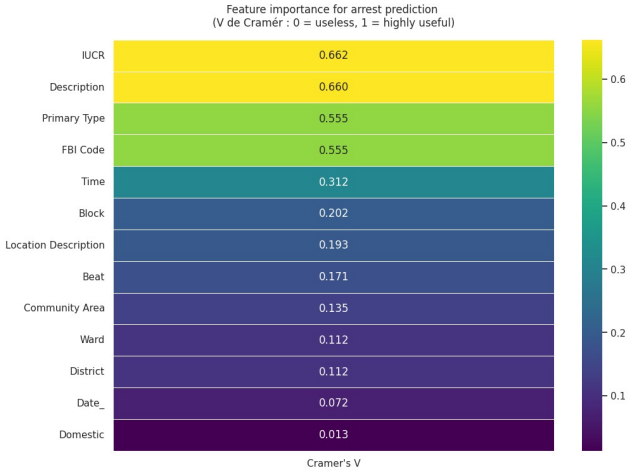
`createOrReplaceTempView("crimes")` gives the DataFrame a temporary SQL name. `spark.sql(...)` then uses `SELECT`, `COUNT`, conditional `CASE`, `AVG`, `GROUP BY`, `HAVING`, and `ORDER BY` to calculate rates by crime type. This is a second way to express a grouped analysis; it is not a separate dataset or independent validation.

Proactive-type versus other reports

The notebook groups Narcotics, Weapons Violation, and Criminal Trespass as "proactive-type" offenses and compares them with all other, mostly victim-reported offenses. The first group is 7.4% of incidents but 40.1% of arrests. Arrests are often the detection mechanism for these offenses, making the target partly circular. The analysis therefore checks model performance separately: 5,383 test incidents in the proactive group have a 62.9% arrest rate; the other 66,707 have a 7.7% rate.

Important wording Say "proactive-type" as the project's grouping label, not as a claim that every such incident was initiated proactively. The grouping is analytical and imperfect.

Feature Screening: What Is Associated with Arrest?



Cramer's V values from the notebook's candidate-feature screen.

How to read the heatmap

- Values near 0 mean weak association; larger values mean stronger association with Arrest.
- Description (0.660) and Primary Type (0.555) rank highly. Time (0.312) is lower; Domestic (0.013) is very weak alone.
- IUCR (0.662) and FBI Code (0.555) are highly associated but redundant with crime descriptions/categories.
- These are association scores, not model coefficients, causal effects, or proof of predictive usefulness.

The calculation in plain language

For each candidate category, Spark's `StringIndexer` maps labels to numeric category IDs. `VectorAssembler` packs them into a feature vector. `ChiSquareTest.test` measures statistical dependence between each categorical feature and the binary Arrest label. The notebook converts the chi-square statistic to Cramer's V using $V = \sqrt{\chi^2/N}$, appropriate here because the target has two classes. NumPy's `sqrt` and `toArray` turn the Spark results into values for a Pandas table and Seaborn heatmap.

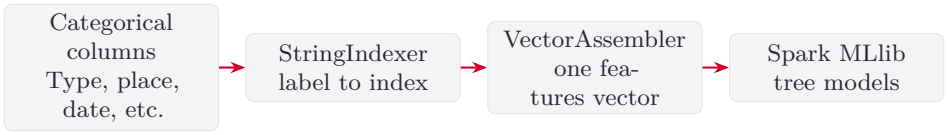
The notebook keeps Primary Type, Description, Location Description, Block, Beat, Ward, District, Date, Time, and Domestic. It excludes ID, IUCR, FBI Code, and Community Area as identifiers or redundant fields. Domestic is retained despite a low standalone association because a feature may contribute in combination with others.

Validation issue The feature screen and its category indexers are fitted on the full dataset before the random train/test split. In particular, Cramer's V uses the target labels across the full data. This creates leakage risk; repeat feature selection and fitting using training folds only.

Feature Engineering and Class Imbalance

From columns to a feature vector

The notebook turns the parsed timestamp into `Date\_` (year-month-day) and `Time` (hour-minute-second). Each feature is cast to text, then a `StringIndexer` maps categories to numeric indices; `handleInvalid="keep"` reserves an index for labels not seen during fitting. `VectorAssembler` packs indexed columns into the single `features` vector expected by the classifiers.



Class weights instead of synthetic records

Only 11.7% of rows have Arrest=True. The notebook assigns each class an inverse-frequency weight:

$$w_1 = \frac{N}{2N_1}, \qquad w_0 = \frac{N}{2N_0},$$

where N is the training-table size, and N_1, N_0 are arrest and no-arrest counts. This gives the two classes roughly equal total weight in training, making missed arrests matter more. The project avoids SMOTE because the workflow uses categorical fields and interpolation between category IDs would create meaningless synthetic values.

Train/test split

`randomSplit([0.7,0.3], seed=42)` randomly assigns about 70% of rows to training and 30% to test; the seed makes the split reproducible. The test set has 72,090 incidents, including 8,497 recorded arrests. Because the split is random, it is not a test on a future year. Related incidents may cross the split, and learned indexers were fitted before splitting; both are limitations.

The Three MLlib Models

All models receive the same `features`, binary `label`, and class `weight`. The notebook sets `seed=42` and `maxBins=3000`; other settings are defaults. A large bin limit accommodates categorical fields with many levels, but this is not hyperparameter tuning.

Model	Beginner's description	Strength / caution
Decision Tree	A sequence of feature-based yes/no splits.	Easy to explain, but a single tree can be unstable and may miss complex patterns.
Random Forest	Many trees trained with randomness; predictions are combined.	Reduces reliance on one tree, usually more stable, but less transparent.
Gradient-Boosted Trees (GBT)	Trees are added in sequence, each focusing on errors left by earlier trees.	Best of these three on the reported arrest-class metrics; still not the best method overall.

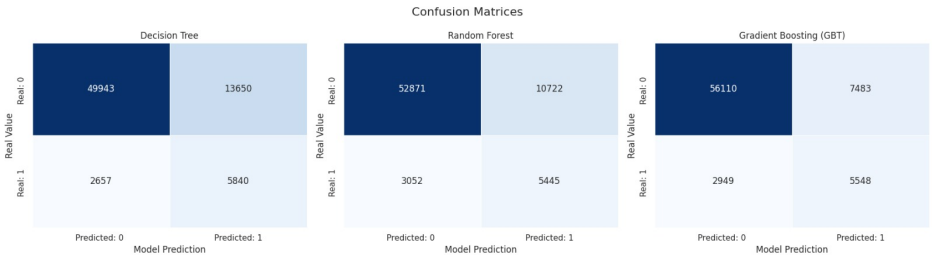
What the code does

`Classifier.fit` learns tree splits from training rows. `model.transform` adds predictions, raw scores, and class probabilities for test rows. `Pipeline` chains learned steps together. The current notebook fits the encoders before the split and models after it; a stronger version would fit the entire preprocessing-and-model pipeline using training data only.

Confusion matrix: count the four outcomes

For each model, the notebook groups predictions by actual `label` and predicted class, then displays a 2 by 2 heatmap. Rows are actual status; columns are predicted status:

	Predicted no arrest	Predicted arrest
Actual no arrest	True Negative (TN)	False Positive (FP)
Actual arrest	False Negative (FN)	True Positive (TP)



The GBT catches 5,548 arrests and produces 7,483 false alarms in this test split.

Evaluation: Which Metrics Matter?

Model	Prec.	Recall	F1	ROC	PR	Plain reading
Decision Tree	.300	.687	.417	.714	.499	Finds more arrests, but raises many false alarms.
Random Forest	.337	.641	.442	.831	.567	Better ranking and fewer false alarms than one tree.
GBT	.426	.653	.515	.844	.635	Best MLlib balance in this comparison.

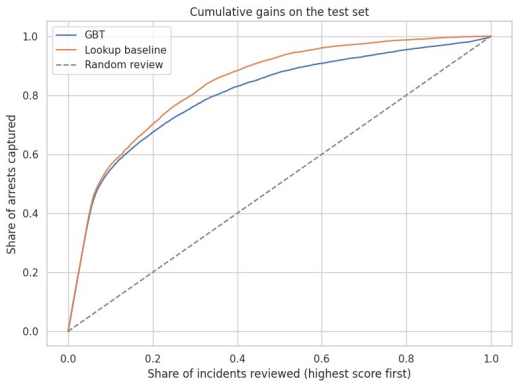
Metric glossary

- **Precision** = TP/(TP+FP): among incidents flagged as arrests, how many actually have a recorded arrest? GBT precision .426 means about 43%.
- **Recall** = TP/(TP+FN): of all recorded arrests, how many did the model find? GBT recall .653 means about 65%.
- **F1** = harmonic mean of precision and recall. It balances the two but does not encode operational costs.
- **AUC-ROC**: ranking ability across classification thresholds; 0.5 is chance-level ranking and 1.0 is perfect ranking. It is not accuracy or probability calibration.
- **AUC-PR**: precision-recall ranking summary, especially useful when positive cases are rare. It depends on class prevalence.
- **Accuracy**: fraction of all rows classified correctly. With an 88.3% majority class, it can look high while recall for arrests is zero.

Two model tables in the notebook The first evaluator reports weighted metrics across both classes; weighted recall equals accuracy, and the majority class dominates the average. A later function explicitly sets `metricLabel=1.0` and reports precision, recall, and F1 for Arrest=True. Those are the class-specific numbers shown in the presentation.

GBT has 7,483 false positives and 5,548 true positives: roughly 1.35 false alarms per recorded arrest found. That is a retrospective test-set count, not an estimate of real operational workload.

The Baseline, Lift, and Robustness



Cumulative gains on the held-out random test set.

The baseline is not a fitted ML model. It is a smoothed lookup table built from training rows, grouped by Primary Type and Description. For a group with k arrests among n training incidents, the score is

$$\frac{k + Mp}{n + M}, \quad M = 10,$$

where p is the overall training arrest rate. This shrinks small-group estimates toward the overall rate. Unseen pairs receive the prior p.

Why this baseline matters

On the same test set, the lookup has AUC-ROC 0.885 and AUC-PR 0.678; GBT has 0.844 and 0.635. The lookup uses only crime type and description, and ranks better than the three tested models. Extra fields such as date, street, beat, and ward did not add measurable lift in this experiment.

The cumulative-gains chart sorts incidents from highest score to lowest. The x-axis is the share reviewed; the y-axis is the share of all arrests captured. At the top 20%, the lookup captures 70% of arrests, GBT captures 67%, and random review captures about 20%. This is a ranking/lift comparison, not a tested staffing plan or causal benefit.

Subgroup robustness

Test group	Rows	Arrest rate	GBT ROC	Lookup ROC
Narcotics, weapons, trespass	5,383	62.9%	.916	.928
All other, mostly victim-reported	66,707	7.7%	.763	.823

The all-crime AUC is partly flattered by easy, partly circular proactive-type cases. Performance falls on the 93% of incidents in the other group, and the baseline still leads.

Reading the Notebook Safely

Five limitations to keep in view

1. **Scope:** one city and one year. Results may not generalize to a different year, city, or reporting system.
2. **Outcome:** Arrest is a recorded event, not guilt, case clearance, or investigative quality.
3. **Time and dependencies:** the split is random, not temporal; timestamps may default to midnight; several incidents may share a case.
4. **Leakage and tuning:** Cramer's V and category indexers are calculated/fitted before the split; models use defaults, without cross-validation or tuning.
5. **Fairness and causality:** geography and offense labels can reflect historical reporting and enforcement. Associations do not establish causes or fair treatment.

A stronger next experiment

Use multiple years and a chronological holdout. Keep incidents from the same case together where possible. Fit encoders, feature screens, and every learned transformation only within each training fold. Tune with cross-validation on training periods, then evaluate once on a later period. Report class-specific metrics, calibration, subgroup error rates, false-alarm workload, and uncertainty. Add richer data only with a lawful, ethical purpose and independent review. Keep a human decision-maker accountable.

Notebook function reference

Function / method	Plain-English meaning
`groupBy().agg()`	Split rows into categories and calculate summaries per group.
`count`, `sum`, `avg`, `round`	Count rows, add values, average a column, and round a result.
`filter`, `orderBy`, `limit`	Keep rows meeting a condition, sort them, and restrict displayed rows.
`join`	Match records from two DataFrames on shared keys, such as Primary Type.
`cast("int")`	Convert Boolean Arrest values into 1/0 so their mean is a rate.
`toPandas()`	Collect a Spark result to local Pandas for plotting; use carefully for large data.
`fit`, `transform`	Learn model/encoder parameters, then apply them to rows.
`evaluate`	Calculate a chosen metric by comparing labels, predictions, or scores.
`twinx`, `scatter`, `heatmap`	Plot with a second y-axis, show paired numeric values, or encode values as color.
`tight_layout`, `annotate`	Prevent plot overlap and add labels to points.

Quick Q&A and Presentation Readiness

Why does the notebook use Spark for 240,000 rows?

Spark is the project's platform and demonstrates an end-to-end distributed workflow. A single-machine library could also handle this one-year dataset; the notebook does not prove Spark is necessary at this scale.

Why is the simple baseline better?

The historical arrest rate by offense type and description already carries a strong signal. The tested extra fields did not improve ranking on this split. This is a result for this dataset and protocol, not proof that ML is never useful.

Does 0.885 AUC mean an 88.5% chance of arrest?

No. AUC-ROC measures ranking across thresholds. It is not an individual probability, percent accuracy, or calibration score.

Does the district adjustment show which district performs best?

No. It adjusts expected counts for crime-type mix, but leaves many other differences uncontrolled. It is a descriptive comparison, not a causal or performance ranking.

Why not use SMOTE?

The project uses categorical predictors and Spark class weights. Interpolating encoded category IDs would create artificial categories without a meaningful interpretation. Class weights increase the training penalty for the minority class instead.

Could this be used to decide where to deploy officers?

Not from this study. The target is a recorded arrest and may reflect past reporting and enforcement. The project has no causal deployment test, temporal validation, fairness audit, or evidence that a score improves safety. Do not use it to label people, deny investigation, or justify coercive action.

What should the audience remember?

Crime type dominates the recorded arrest pattern; context and measurement matter; the tested MLlib models do not beat a transparent type-by-description baseline; and a recorded-arrest score must never replace human judgment.

A final presentation habit For every percentage, state what is in the numerator and denominator. For every model score, name the test set and target. For every recommendation, say what the data do not establish.

Prepared from the project Databricks notebook and accompanying README. Counts and model metrics are reported as documented in those materials.